

TRADING AGENT: AN EVIDENCE-FIRST MULTI-AGENT AI SYSTEM FOR NSE EQUITY RESEARCH AND EXECUTION

**PROJECT REPORT FOR THE COURSE
AD23731 FOUNDATIONS OF AGENTIC AI**

Submitted by

M HARISH (Reg. No. 231501058)

AFRAH M (Reg. No. 231501008)



**DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, CHENNAI, 602105**



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the project work done by **M Harish (Reg. No. 231501058)** and **Afrah M (Reg. No. 231501008)** of VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in charge

Submitted for the Project Review held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

INDEX

S. NO.	TITLE	PAGE NO.
1.	Introduction	1
2.	Problem Statement and Business Case	2
3.	User and Stakeholder Analysis	2
4.	Proposed Agentic AI Solution	3
5.	Dataset and Knowledge Sources	5
6.	Tools and Technologies Used	6
7.	Agent Architecture	6
8.	Agent Workflow and Orchestration	8
9.	Agentic RAG Implementation	9
10.	System Implementation	10
11.	Experimental Evaluation and Results	13
12.	Comparison, Innovation and Scalability	15
13.	Conclusion and Future Work	16
14.	References	17

1. INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Retail participation on the National Stock Exchange (NSE) has grown quickly, and market data is now easy to obtain: ten years of daily prices, index membership lists and company announcements can be downloaded by anyone. What an individual investor still lacks is the process that sits between the data and the trade. In a fund, an idea passes a research note, a risk check, a cost estimate and an execution desk before money moves. A retail investor usually has the idea and places the order a minute later.

Trading Agent (shown in its dashboard as LinearTrade) brings that process to NSE cash equities. It is a multi-agent AI platform built around one rule: the AI agents may recommend a trade, but only deterministic code can approve one. Every number it reports is dated, every charge comes from the schedule in force on that day, and every strategy is judged on data it has never seen.

1.2 NEED FOR AGENTIC AI

Language models are strong at reading and arguing over evidence, and weak at arithmetic and at knowing what date it is. An agentic design, where specialised agents each do one job and pass typed results to each other, uses the first strength and guards against the second:

- **Different evidence needs a different reader:** chart structure, business quality and news tone are separate tasks, so each gets its own agent, prompt file and output schema.
- **Disagreement surfaces weak arguments:** a bull and a bear argue for two rounds and each must answer the other's last point before a moderator decides.
- **Research needs live data, not model memory:** the research desk plans which data tools to call and must quote what they return, with a date and a source.
- **Authority stays with code:** a model can be persuaded or misled by text in the news it reads, so the final yes or no is ordinary code that runs after the agents finish.

1.3 PROJECT OBJECTIVES

Table 1.1: Project objectives and their pass conditions

No.	Objective	Pass condition
O1	Trustworthy data	Index membership by date, bonus and split adjustment, dated Indian charge schedules in every result
O2	Honest measurement	Walk-forward testing on unseen data, after all charges, against NIFTY 50
O3	Model kept in check	Typed BUY / SELL / HOLD outputs, a veto in code outside the agent graph, a kill switch
O4	Grounded research	A tool-using research agent whose every number traces to a dated tool result
O5	Safe operation	Decimal money, repeat-safe order keys, a full journal, reconciliation with the broker
O6	Out-of-sample performance	A fixed candidate, chosen on 2015-2024 data, tested once on untouched 2025-26 data

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 PROBLEM DEFINITION

Build a research and trading platform for NSE cash equities in which every reported number can be defended, the language model helps with research but never has the power to trade, and capital is committed only after a strategy has proved itself on data it has never seen.

Home-made backtests fail silently in four ways, and each one makes results look better while producing no visible error. The platform fixes each one mechanically:

Table 2.1: Four silent biases and the fix applied to each

Bias	What goes wrong	Fix in this system
Same-day fill	Buys at the close that produced the signal	Fill at the next session's open
Charges left out	STT, stamp duty, GST and DP fees ignored	Dated charge schedule applied per trade
Survivorship	Today's NIFTY 50 tested over the past	Index membership checked by date
Unadjusted prices	A 1:1 bonus reads as a 50% crash	Corporate actions applied before any indicator

2.2 BUSINESS AND REAL-WORLD CONTEXT

Charges are large and set by law. With the project's verified charge model, buying and later selling a ₹1,00,000 delivery position at Zerodha costs ₹237.82 (23.8 basis points), of which STT alone is ₹200. Rates have changed over the years, so four dated schedules (2015, 2020, 2024 and March 2026) are kept and the one in force on each simulated day is applied.

Regulation shapes the design. SEBI's 2025 framework for retail algorithmic trading treats an algorithm whose logic is disclosed and repeatable (white box) differently from a hidden one. Because the order decision is fixed code and the model only advises, the trading logic stays white box even when a prompt changes. Mistakes are also asymmetric: a duplicate order costs money at once, while a skipped trade costs nothing measurable, so whenever the system is unsure it declines to act.

3. USER AND STAKEHOLDER ANALYSIS

3.1 TARGET USERS

- **Researcher:** wants to know whether a stock or strategy deserves attention, with an answer that could be proved wrong. Uses the research desk, the agent debate and the Backtest Studio.
- **Operator:** runs the system on a schedule and needs to know what it did and why. Uses the Cockpit, the Portfolio Monitor and the scheduler, with every action journalled.
- **Reviewer:** an examiner, an auditor or the same user months later, who must reconstruct a past decision without re-running it. Uses the journal, saved signals and the desk ledger.

3.2 STAKEHOLDER IDENTIFICATION

Table 3.1: Stakeholders and how the system serves each

Stakeholder	What they need	How the system provides it
Retail investor / researcher	A reliable answer on whether a strategy is worth it	Walk-forward testing, efficiency, sourced research answers
Operator	Predictable, auditable automation	NSE-calendar scheduling, Redis session locks, a journal of every action
Broker (Zerodha Kite)	Correct orders, no duplicates, rate limits respected	Repeat-safe order keys, rate limiting, reconciliation
Regulator (SEBI)	Disclosed, repeatable order logic	Model kept out of the order path; decisions in fixed code
Examiner / auditor	Decisions that can be reconstructed	Saved rejected signals, token use per agent, prompts in git

3.3 USER REQUIREMENTS

Table 3.2: Functional requirements and where they are met

ID	Functional requirement	Where it is met
FR1	Load NSE daily prices adjusted for bonuses and splits	market_data/ (corporate actions, bar store)
FR2	Limit any past stock list to the members on each date	market_data/universe.py
FR3	Compute indicators without future data	features/pipeline.py
FR4	Run a multi-agent debate that ends in a typed verdict	agents/graph.py, agents/schemas.py
FR5	Answer research questions from dated tools, with sources	agents/research_chat/
FR6	Reject signals that fail risk rules; size by risk	risk/veto.py, risk/position_sizing.py
FR7	Send orders safely and reconcile with the broker	execution/router.py, idempotency.py, reconciler.py
FR8	Test strategies walk-forward and grade agents after the close	backtest/walk_forward.py, post-close review

Non-functional requirements: money is stored as Decimal, never float; the domain package imports nothing and CI enforces the layer rules; agent responses are cached by input hash so runs are reproducible; anything missing or unreadable (a verdict, the kill-switch state) is treated as a refusal; and every rejected signal is saved, because rejections are the record of what the system chose not to do.

4. PROPOSED AGENTIC AI SOLUTION

4.1 OVERALL SOLUTION

The system runs in four stages, cheapest first: arithmetic screens 580 stocks down to ten, seven agents debate those ten, a code veto runs six checks after charges, and an approved signal becomes one idempotent order. Figure 4.1 shows how the parts fit together.

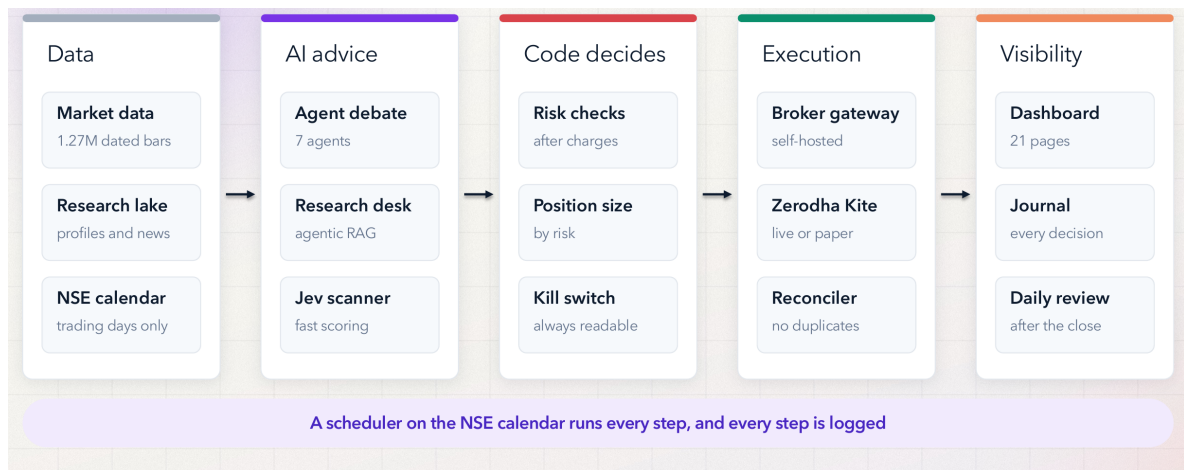


Figure 4.1: System architecture: data in, AI advice, code decides, execution, visibility

4.2 KEY FEATURES

- **Dated price store:** 1,265,794 adjusted daily bars for 580 NSE stocks (January 2015 to September 2026) in Parquet, queried with DuckDB.
- **Verified Indian cost model:** brokerage, STT, exchange and SEBI fees, stamp duty, GST and DP charges as dated schedules, plus square-root slippage.
- **Seven-agent debate:** LangGraph on Microsoft Foundry; every agent returns a validated Pydantic object, and a failed agent is recorded by name.
- **Agentic RAG research desk:** a planning model with eight data tools that dates and sources every number it writes.
- **Code veto and kill switch:** reward-to-risk recomputed after costs, stop-side check, limits per stock, sector and day, and a kill switch in Redis.
- **Jev scanner:** a fast second opinion that scores a point-in-time snapshot in about 280 ms and sizes from a measured edge.

4.3 AGENT-BASED APPROACH

The debate layout follows TradingAgents (Xiao et al., 2024): analysts, a bull-bear debate, a moderator and a risk review. Six details were changed for a system that handles money, each recorded in ADR 0004:

Table 4.1: Changes made to the reference multi-agent trading design

Aspect	Reference design	This project
Output	Free text, parsed by a second model call	Pydantic structured output from every agent
Prompts	Strings inside code	Markdown files in agents/prompts/, diffable in git
History	One long string re-sent every turn	Typed DebateTurn records; bounded token cost
Final say	A model acting as risk judge	Risk manager advises; the veto is code
Reward to risk	Compared before costs	Recomputed after costs on both sides
Failures	A failed agent stops the run	Failure recorded by name; graph continues

The after-cost check proved itself on a live run: the moderator proposed a trade at 1.85 reward to risk before costs, which falls to 1.76 after a 12-basis-point round trip, below the 1.8 floor. The code veto caught it.

4.4 JEV SCANNER

Alongside the debate, the Jev scanner gives a fast second opinion. It builds a point-in-time snapshot from daily bars with the symbol name hidden, asks Jev (model jev-1.13.0) two fixed questions about the volatility regime, and clears a trade only if every score reaches its 0.50 gate. Risk is a quarter-Kelly fraction of the measured edge, capped at 1% of capital, and a Foundry model audits each scan against its inputs (Figure 4.2).

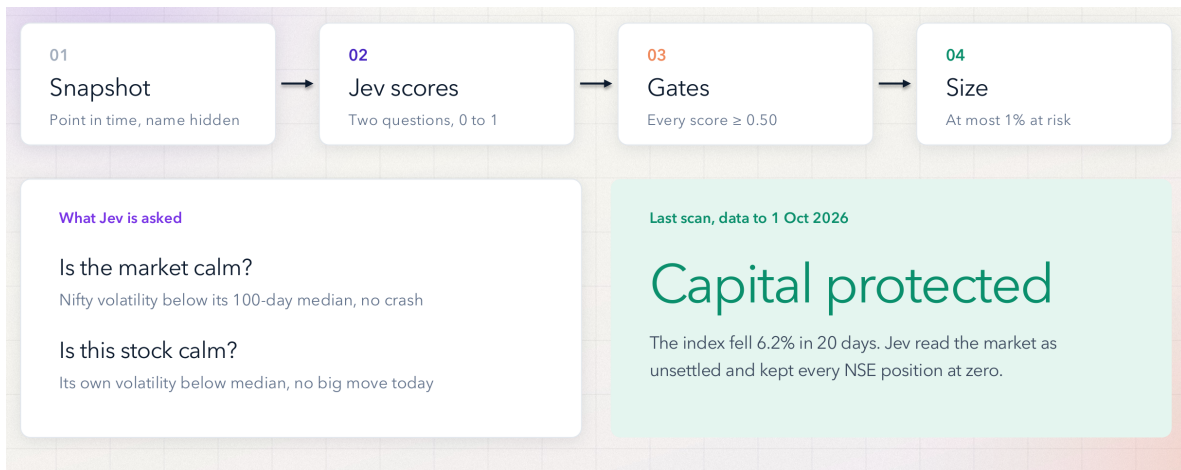


Figure 4.2: Jev scanner: snapshot, scores, gates, size

5. DATASET AND KNOWLEDGE SOURCES

5.1 DATASET AND KNOWLEDGE SOURCE DESCRIPTION

Table 5.1: Datasets and knowledge sources

Source	Contents and coverage	Used for
NSE daily prices	580 stocks, 1,265,794 adjusted bars, 2015 to 2026	Indicators, screens, backtests, debate inputs
Index membership history	NIFTY 50 (86 names) and liquid 200 (579 names) with start and end dates	Eligibility on each date
Corporate actions	24 splits and bonuses with dates and ratios	Price adjustment before indicators
NSE trading calendar	Holiday list rebuilt from market-wide closed days	Jobs on exchange days only
Dated charge schedules	Four Zerodha schedules (2015, 2020, 2024, 2026)	Charges on every simulated and real trade
Research knowledge lake	284 company profiles, 1,420 persona cards, 2,759 news items, 20 model baskets	Grounds the research desk
Fundamentals store	ROCE and promoter holding with period and publication date kept apart	Quality checks without future data
Desk ledger	Dated debates, trade reviews, scorecards and flags	Long-term memory for the research agent

5.2 POINT-IN-TIME HANDLING

Membership checked by date removes survivorship bias: the NIFTY 50 panel grows to 82 stocks over the study and the liquid 200 panel to 562, each tradable only while it was in the

index. Every fundamental figure stores both the quarter it describes and the day it was published, so a 2018 backtest never sees later ratios. Lake evidence enters a debate only when the debate date is after the lake's export date.

6. TOOLS AND TECHNOLOGIES USED

Versions are as installed in production on 3 October 2026. Each choice has one reason; the architecture decisions are recorded as ADRs in docs/adr/.

Table 6.1: Tools, versions and the reason for each

Layer	Technology (version)	Why this choice
Language	Python 3.12, strict typing	One typed language for agents, quant code and API
Agent framework	LangGraph 1.2, langchain-core 1.6	State graph with merge rules for parallel agents
Model access	Microsoft Foundry via langchain-openai, azure-identity	One endpoint, a deployment per agent (ADR 0003)
Models	gpt-5-mini default; per-role overrides	Swap models by configuration, not code
Validation	Pydantic 2	Agent outputs and tool arguments are schemas
Price store	Parquet (PyArrow) + DuckDB	Fast scans over ten years of bars (ADR 0002)
Analytics	pandas, pandas-ta, SciPy	Indicators, metrics, statistics
Database	PostgreSQL 16, SQLAlchemy 2 async, Alembic	Signals, orders, journal; versioned migrations
Cache and locks	Redis 7	Response cache, session locks, order keys, kill switch
API	FastAPI 0.141, Uvicorn, Server-Sent Events	Typed HTTP API, streamed research answers
Scheduling	APScheduler on the NSE calendar	Nothing runs on an exchange holiday
Dashboard	Next.js 16.3, React 19, Tailwind 4	22 pages, server-rendered
Broker	Zerodha Kite through a self-hosted gateway	Unmodified container, official SDK (ADR 0001)
Quality	ruff, mypy strict, import-linter, pytest, ESLint, tsc	Every gate must pass before a deploy
Deployment	Docker Compose, GitHub Actions, Azure VM, Tailscale	Reproducible images, private network

7. AGENT ARCHITECTURE

7.1 AGENT COMPONENTS

Table 7.1: Agents in the trading debate

Agent	Input	Output	Authority
Technical analyst	15 bars with SMA 20/50/200, ATR 14, RSI 14	AnalystOpinion	Advice
Fundamental analyst	Dated fundamentals and lake evidence	AnalystOpinion	Advice
Sentiment analyst	Market context and dated news	AnalystOpinion	Advice
Forensic auditor	Beneish M-Score and Altman Z-Score evidence	Audit note	Advice
Bull researcher	All opinions and the bear's last argument	DebateArgument	Advice
Bear researcher	All opinions and the bull's last argument	DebateArgument	Advice

Agent	Input	Output	Authority
Moderator	Typed debate history, list of failed agents	ModeratorVerdict	Proposes action and levels
Risk manager	The moderator's verdict	RiskVerdict	Advice

7.2 TOOLS

The two agent systems use tools in opposite ways on purpose. Debate agents call no tools: the pipeline gathers all evidence before the graph starts, so a replay shows the agents exactly the same inputs. The research desk does use function calling, because a free question cannot be planned in advance. Its eight tools (screener, browser, comparison, persona, research, sector, portfolio and memory) are plain arithmetic over dated data and return JSON with an as-of date and a source, or null with a reason.

7.3 MEMORY

- **Working memory:** DebateState, a typed dictionary passed through the graph; analysts run in parallel, so each field has a merge function that gives the same result in any finishing order.
- **Episodic cache:** Redis stores each model response under a hash of deployment, schema and messages, which makes agent tests repeatable.
- **Long-term memory:** PostgreSQL keeps every signal, including HOLDs and vetoes, with the journal and token use per agent; the desk ledger lets the research agent recall earlier conclusions.

7.4 REASONING FLOW

Figure 7.1 shows what each step receives and hands on. Analysts start together, the bull and bear argue for two rounds, the moderator gives BUY, SELL or HOLD with entry, stop and target, and the risk manager comments. Only state crosses the boundary into code; the graph never emits an order.

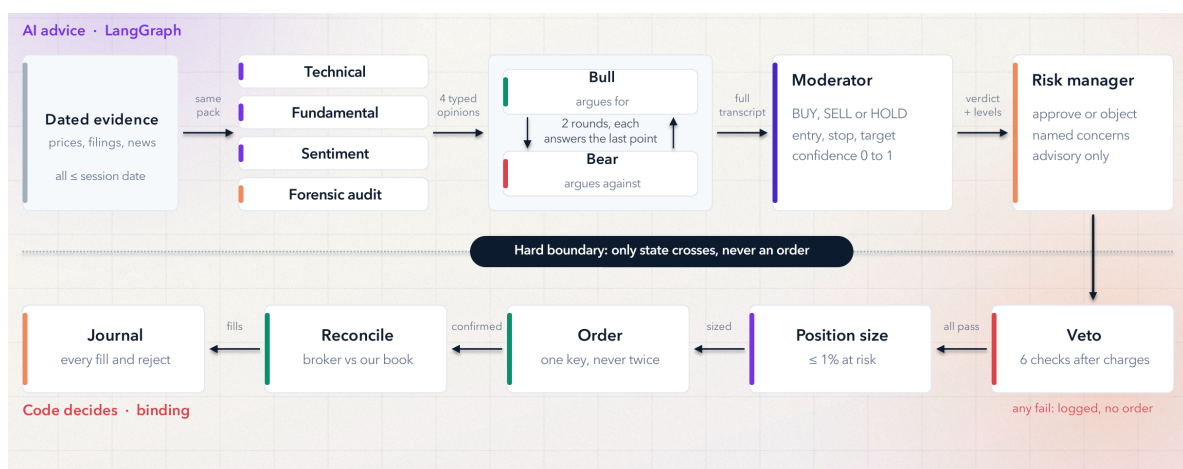


Figure 7.1: Inside the agent debate, and the code that decides after it

The veto recomputes the trade rather than trusting the model. With entry E, stop S, target T and round-trip cost C per share, net reward-to-risk is $R = ((T - E) - C) / ((E - S) + C)$. A signal

passes only if $R \geq 1.8$, the gain covers at least three times the cost, the stop is on the correct side and confidence clears its floor. Size is set by risk, never by confidence: a stop-out costs at most 1% of the account including charges, and no position exceeds 25%.

7.5 AGENT INTERACTIONS

- **Debaters answer each other:** each researcher sees the other's last argument and records what it concedes.
- **Failure is recorded, not hidden:** a blocked or malformed agent raises `AbstentionError`; later agents are told which voices are missing.
- **Quota is shared:** at most four concurrent calls per Foundry deployment; retries honour `Retry-After`.
- **The graph cannot approve itself:** it returns state, not a trade; a separate function turns it into a signal, and a `HOLD` or missing level creates none.

8. AGENT WORKFLOW AND ORCHESTRATION

8.1 WORKFLOW DESIGN

A trading day is a set of scheduled jobs on the NSE calendar (Figure 8.1). Research starts at 08:15, an hour before the open; entries are placed at 09:20, once the account balance is meaningful; the position monitor checks stops and targets every minute; and the review job runs at 19:00 after the day's prices are final.

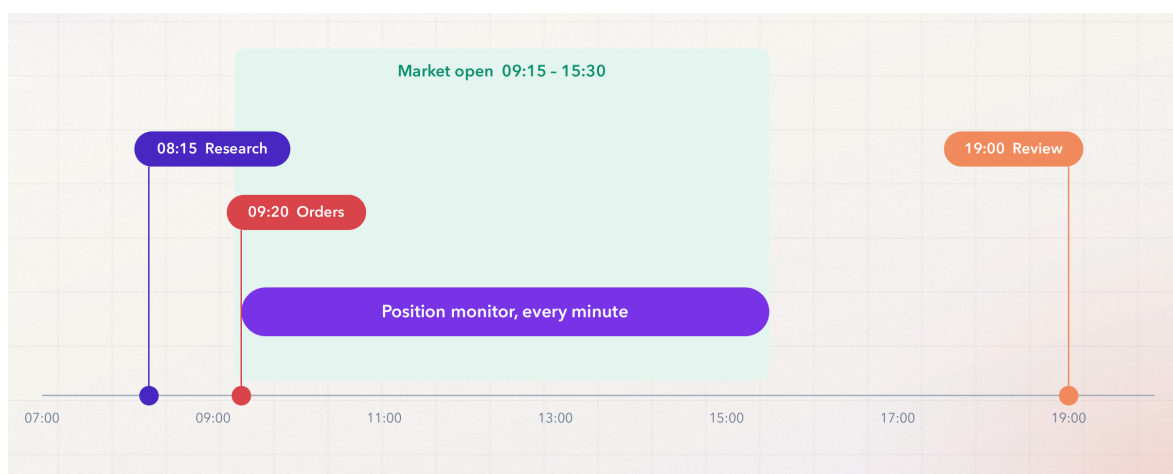


Figure 8.1: One trading day on the NSE calendar (IST)

8.2 TASK DECOMPOSITION

The daily job is split into small, separately tested steps ordered by cost (Figure 8.2). Screening is free arithmetic; only the debate spends tokens, about 19,000 per stock, so debating the top ten instead of all fifty saves roughly 80% for the same decisions.

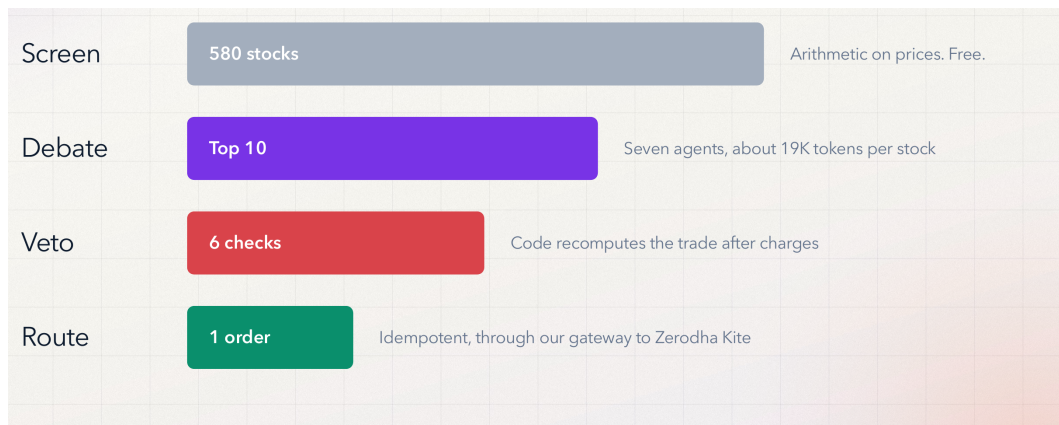


Figure 8.2: Four stages, cheapest first

8.3 AGENT COORDINATION

No agent assigns work to another, so there is no coordinating model to argue into a mistake. Inside the graph a fixed layout and one round counter set the order and merge functions combine results. Between stages typed objects are handed on: DebateState becomes a TradeSignal, then a sizing result, then an Order, each by a pure, tested function. Between processes a Redis session lock stops two copies from running the same session.

8.4 ORCHESTRATION IMPLEMENTATION

The scheduler wraps APScheduler and derives run times from the exchange calendar. Each pipeline is a plain class that receives its dependencies, so it is tested without a scheduler, broker or model. Live trading requires an explicit confirmation flag; without it the order router refuses to send anything, and the system runs in paper mode by default.

9. AGENTIC RAG IMPLEMENTATION

9.1 RETRIEVAL ARCHITECTURE

There are two retrieval paths, and neither can see data published after the day in question (Figure 9.1). The research desk is agentic: a planner on Foundry chooses among eight dated tools, may call several at once, and loops up to six rounds before answering. The trading debate is fixed to the date: the pipeline, not the model, runs date-limited queries, so a replay is exact.

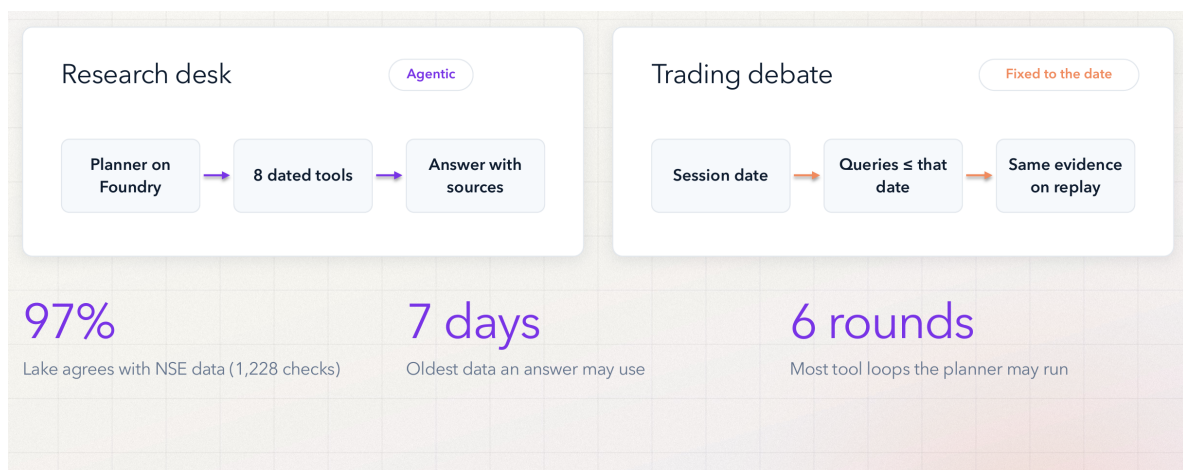


Figure 9.1: Two retrieval paths: agentic research desk and date-fixed debate

9.2 KNOWLEDGE RETRIEVAL

Retrieval is structured rather than similarity search, because most questions are about numbers and comparisons, such as ROCE above 15% for five years. Prices come from DuckDB over Parquet with rows after the session date excluded inside the query; membership from a dated history; fundamentals filtered by publication date; peers by sector median and percentile; and earlier desk conclusions from the ledger. Each result carries the measured track record of its signal type.

9.3 GENERATION PROCESS

The model's output is controlled at four points. The loop has a fixed budget: the last 16 messages, tool results cut at 14,000 characters, and after six tool rounds a final answer with tools turned off. The system prompt sets evidence rules: every number must come from a tool result in the conversation, sources are dated, and coverage gaps are named. Debate agents are bound to strict Pydantic schemas that reject unknown fields. Finally, any price a model proposes is only an input to the veto.

9.4 RETRIEVAL-QUALITY EVALUATION

Table 9.1: Retrieval quality evaluation

Property	How it was checked	Result
No future data in indicators	Assertion in the feature pipeline, unit-tested	Passing
No future data from the lake	Lake evidence withheld before its export date	Enforced
Correct index membership	Dated history; 50 grows to 82, 200 to 562	Verified
Stale or short history	At most 7 days old, at least 200 bars	Enforced
Agreement between sources	Lake signal prices against NSE closes	97% of 1,228 stocks match
Grounded answer, live run	Three-company margin comparison	1 tool call, 15,204 tokens, every figure matches the snapshot

10. SYSTEM IMPLEMENTATION

10.1 FUNCTIONAL PROTOTYPE SCREENSHOTS

The screenshots below were captured from the production deployment on 3 October 2026.

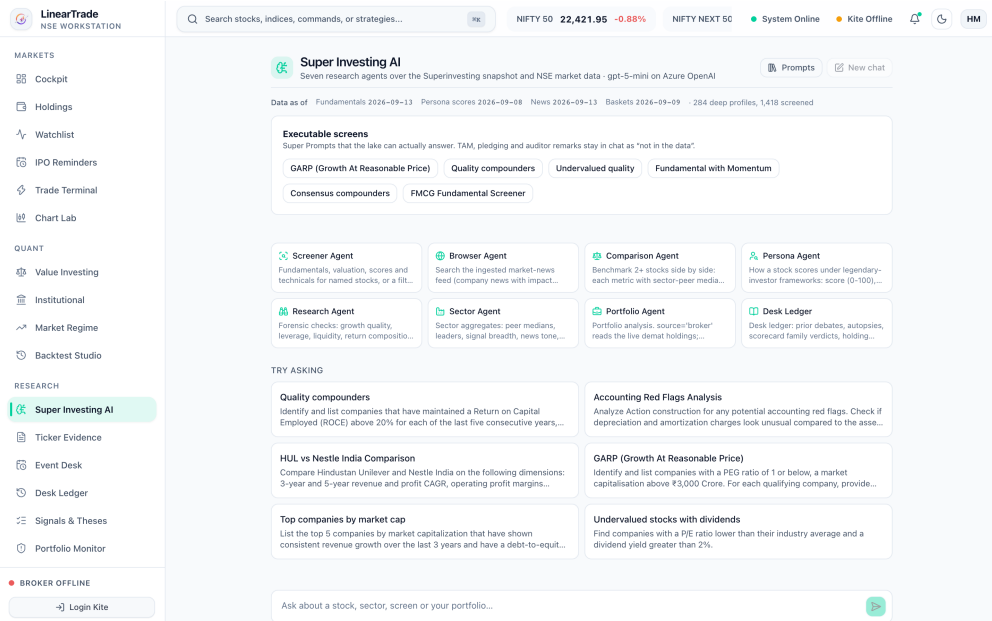


Figure 10.1: Super Investing AI research desk with data dates and eight tool agents

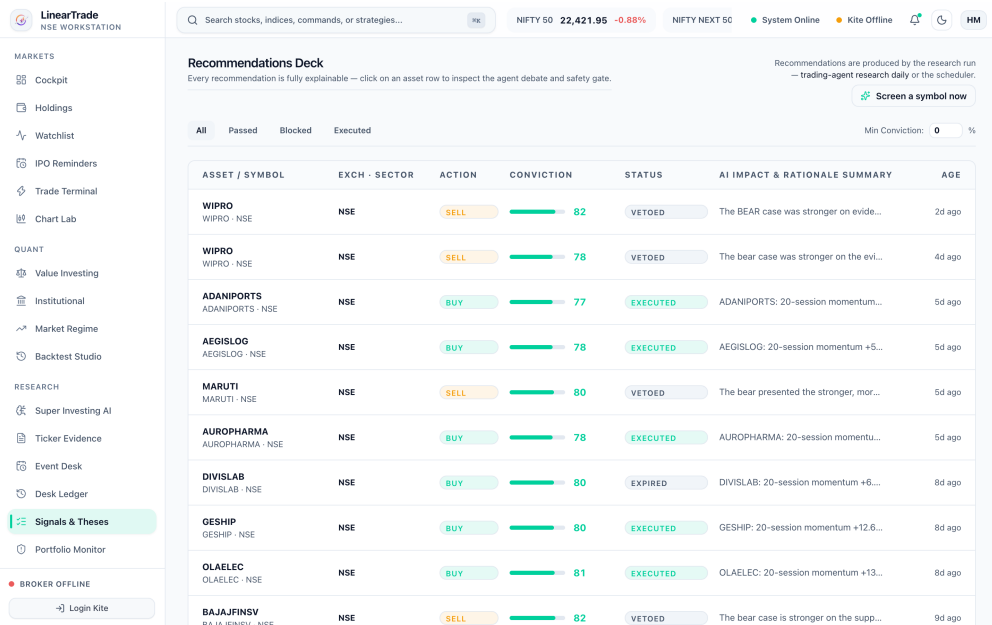


Figure 10.2: Recommendations deck: each verdict with its veto status

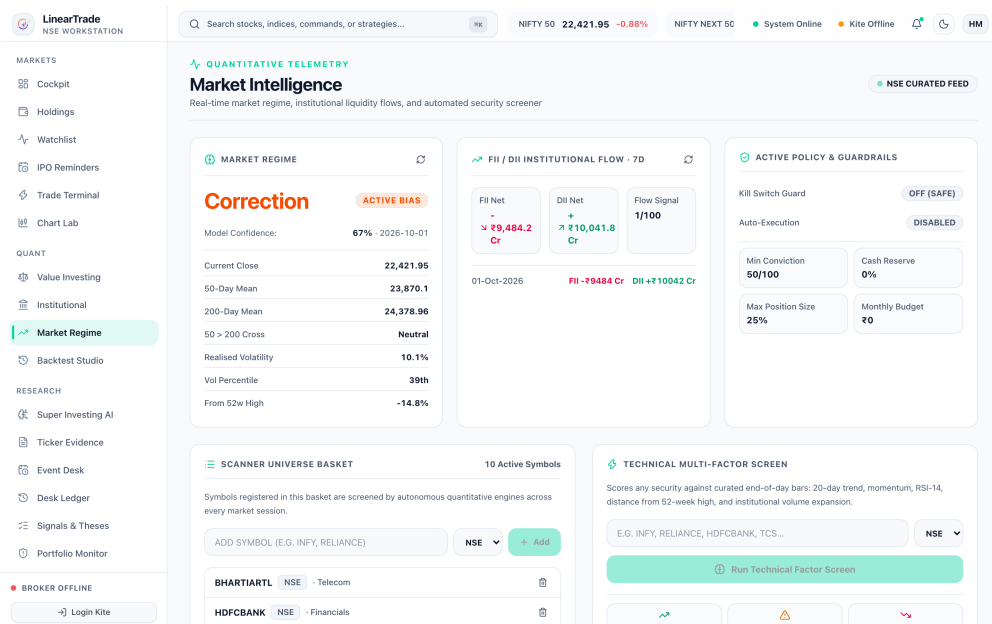


Figure 10.3: Market intelligence: regime, institutional flows and guardrails

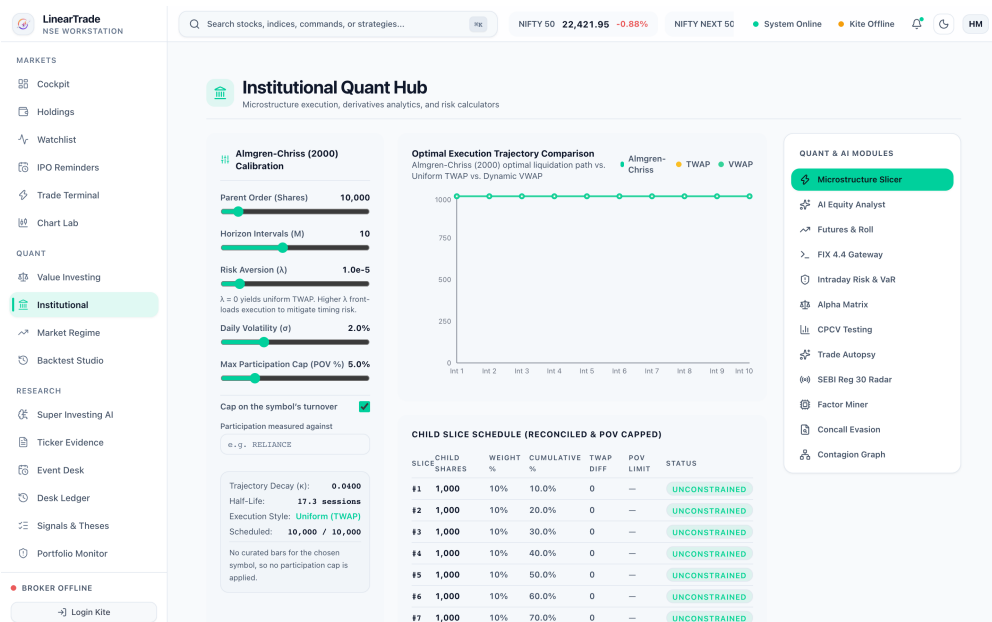


Figure 10.4: Institutional quant hub: execution scheduling and risk modules

10.2 CLOUD DEPLOYMENT

Every push to the main branch runs CI (secret scan, dependency audits, lint, strict types, layer rules and tests); only when all pass are the API and web images built and deployed over SSH to an Azure VM running the full stack in Docker (Figure 10.5). No port is public: the dashboard is reachable only over a private Tailscale network.

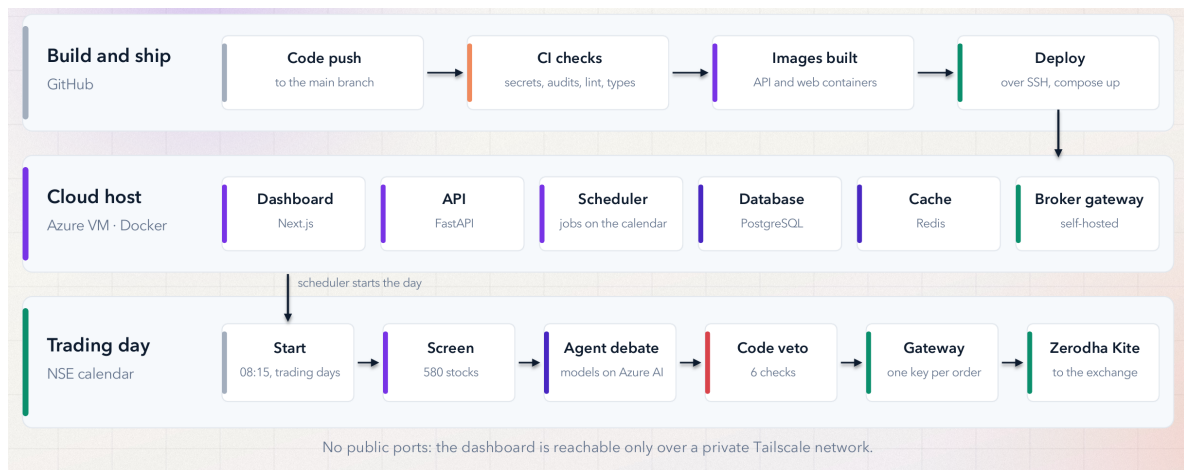


Figure 10.5: From cloud to live order: build, host, trading day

10.3 SOURCE-CODE REPOSITORY

Table 10.1: Implementation summary

Item	Value
Repository	github.com/HarishM-AIML-A-58/Trading-Agent (branch master)
Python source	248 files, 55,153 lines in 15 layered packages
Agent prompts	11 Markdown files under agents/prompts/
Interfaces	Typer CLI; FastAPI with 15 route modules; Next.js dashboard with 22 pages
Tests	192 passing backend tests, 34 Jev scanner tests, 4 Redis integration tests
Design records	5 ADRs in docs/adr/ and runbooks for login, deployment and cost checks

Packages follow strict layers: domain, costs, market_data, features, strategy and backtest at the base; then agents, risk, execution, persistence, orchestration, api and cli. Risk sits above agents, so agent code cannot import, weaken or skip the risk checks, and an import-linter rule in CI keeps it that way.

11. EXPERIMENTAL EVALUATION AND RESULTS

11.1 EVALUATION METRICS

Table 11.1: Evaluation metrics

Metric	Why it is reported
Return and CAGR	Growth comparable across periods, shown next to NIFTY 50
Sharpe and Sortino ratios	Return per unit of total and downside risk
Maximum drawdown	The largest fall from a peak
Walk-forward efficiency η	Unseen-data return divided by design-data return; near 1.0 means the settings generalise
CPCV paths	Combinatorial purged cross-validation across 15 paths
Hit rate and edge in R	For Jev: share of trades reaching target, and average result in risk units
Tool calls, tokens, sourced figures	Cost and faithfulness of the research agent

11.2 TESTING METHODOLOGY

Strategies were tested in rolling walk-forward windows (24 months to choose settings, the next 6 to test) from 2015 to 2026, starting with ₹10,00,000, filled at the next open with dated

charges, spread and square-root impact, on panels checked by date. Study 2 chose one candidate using 2015-2024 only and tested it once on an untouched holdout. Study 3 calibrated Jev on point-in-time snapshots every 10 sessions over three years, with symbols and dates withheld; a win is a 3 ATR gain before a 1.5 ATR loss within 20 sessions. Software is tested by the full pytest suite, with reference backtests locked to the rupee.

11.3 PERFORMANCE RESULTS

Table 11.2: Study 1: unseen-data results across 20 walk-forward windows, after charges

Panel	Strategy	Design data	Unseen data	η	Sharpe
Liquid 200 (562 by date)	Volatility-adjusted momentum, top 30	+140.66%	+134.22%	0.95	+0.24
Liquid 200	12-1 momentum, top 15	+98.06%	+106.33%	1.08	+0.14
Liquid 200	Capped equal-weight control	+50.18%	+21.71%	0.43	-0.14
NIFTY 50 (82 by date)	Volatility-adjusted momentum, top 15	+79.80%	+88.41%	1.11	+0.05
NIFTY 50	Equal-weight control	+146.66%	+76.76%	0.52	-0.02

The best strategy returned +134.22% on data it never saw, more than six times its equal-weight control (+21.71%), with an efficiency of 0.95: it kept pace with its design-period result. On both panels the momentum strategies beat their controls, and two of them did better on unseen data than in design (η above 1).

Table 11.3: Study 2: fixed candidate (12-1 momentum, top 30, quarterly, liquid 200)

Measure	Candidate	NIFTY 50
Walk-forward return, 16 windows, 2015-2024	+192.39%	+189.07%
Profitable windows; walk-forward Sharpe	11 of 16; 0.57	—
CPCV median Sharpe, 15 paths	1.10	—
Holdout return, 1 Jan 2025 to 9 Sep 2026, after tax	+8.36%	-1.39%
Holdout round trips; charges and tax	57; ₹5,095 and ₹1,775	—

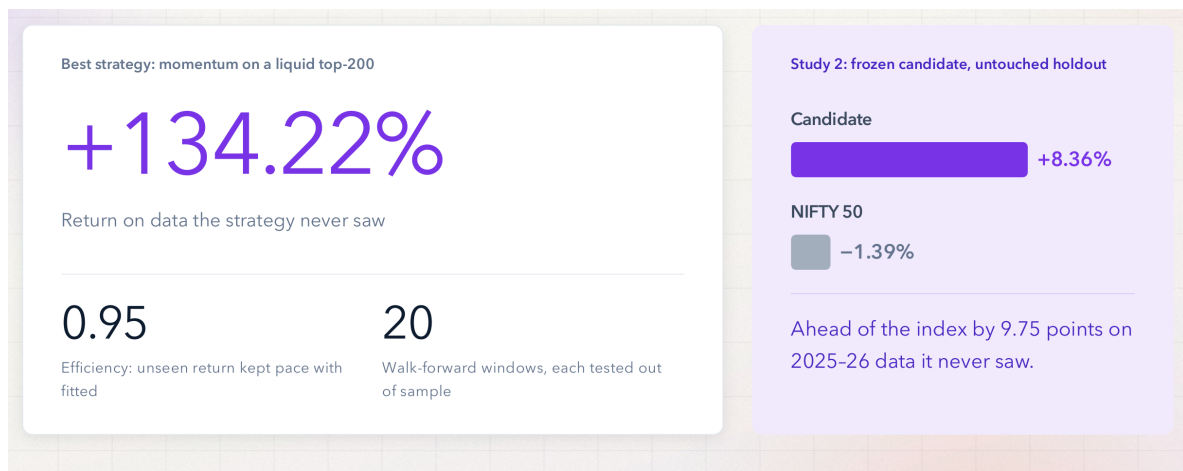


Figure 11.1: Study 1 best strategy and Study 2 holdout against NIFTY 50

Study 3 calibrated Jev's questions. The first version, inherited from the original scanner, cleared one GO in 1,200 samples; the second showed that only volatility questions separated

outcomes; the third asks about the market and stock volatility regime. Jev's probabilities match the deterministic rule its questions describe on 94% of equity and 99% of ETF samples.

Table 11.4: Study 3: Jev v3 GO calls on unseen data

Test set	Samples	Result
Main holdout	280	31% hit rate against 22% base; +0.20R per GO trade; edge +0.28R
12 stocks never used in design	720	Edge +0.11R over taking every trade
All unseen data pooled	1,300 (402 GO)	Edge +0.10R over taking every trade

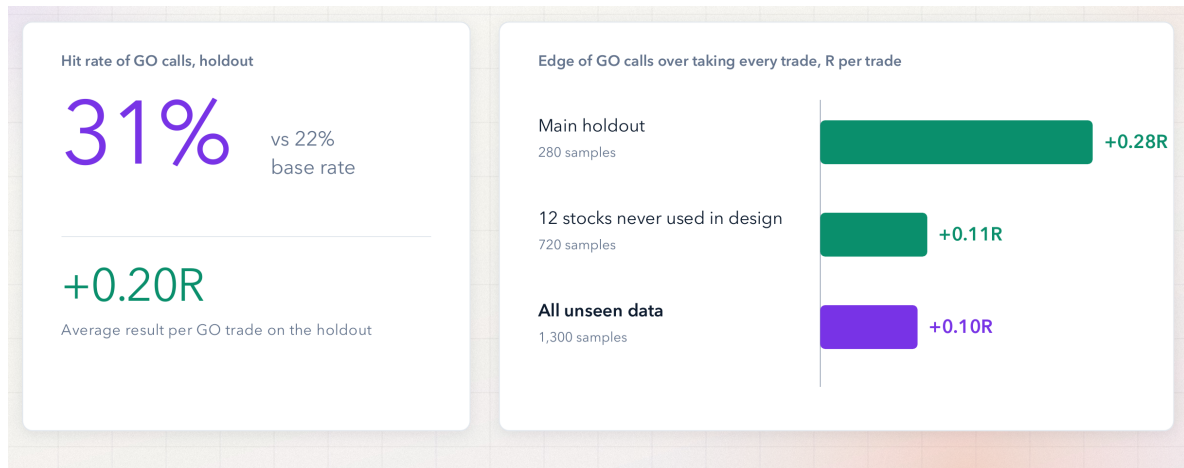


Figure 11.2: Jev's questions pick better trades on every unseen test set

Software: the backend suite passes in full (192 tests), along with 34 Jev scanner tests and 4 Redis integration tests run against a real Redis. Lint, strict type checks, layer rules and the dashboard build all pass in CI, which gates every deployment.

11.4 SUCCESS-METRIC ACHIEVEMENT

Table 11.5: Objectives and achieved outcomes

Objective	Achieved outcome	Status
O1 Trustworthy data	Dated membership, 24 corporate actions, four dated charge schedules	Achieved
O2 Honest measurement	20 walk-forward windows after charges; efficiency 0.95 for the best strategy	Achieved
O3 Model kept in check	Typed outputs; six-check code veto; kill switch that fails safe	Achieved
O4 Grounded research	Every desk figure traced to a dated tool result in live runs	Achieved
O5 Safe operation	Decimal money, repeat-safe keys, journal, reconciliation; 192 tests passing	Achieved
O6 Out-of-sample performance	Holdout +8.36% against -1.39% for NIFTY 50	Achieved

12. COMPARISON, INNOVATION AND SCALABILITY

12.1 COMPARISON WITH BASELINE

Table 12.1: Comparison with a typical retail backtest, a single LLM and TradingAgents

Dimension	Retail backtest	Single LLM	TradingAgents	This system
Results on unseen data	Rarely	No	Not on Indian data	20 windows + holdout
Charges	Flat or none	Ignored	Not modelled	Dated statutory schedules

Dimension	Retail backtest	Single LLM	TradingAgents	This system
Stock list	Today's members	—	Fixed tickers	Membership by date
Output	Numbers	Free text	Free text, parsed	Validated schemas
Final decision	User	Model	Model risk judge	Code veto after costs
Order safety	Manual	None	Simulated	Repeat-safe keys, reconciliation

12.2 ADVANTAGES OF THE PROPOSED SYSTEM

- **Verifiable evidence trail:** every thesis links to dated, sourced evidence and a saved transcript.
- **Bias surfaced by design:** the bull-bear debate stress-tests each idea before a verdict.
- **Arithmetic in code:** the veto, sizing and charges are deterministic, so limits hold every time.
- **Real economics:** statutory Indian charges in every result keep paper profits honest.

12.3 INNOVATION

- **A formal advise-decide boundary:** agents and code are separated by package layers enforced in CI, so model output can never become an order directly.
- **After-cost veto:** reward to risk is recomputed with charges on both sides, a check the reference design omits.
- **Calibrated fast opinion:** Jev's questions are tuned on labelled outcomes with names hidden, and sizing follows the measured edge.
- **Date-grounded agentic RAG:** structured, dated retrieval with a measured track record attached to every result.

12.4 SCALABILITY

Cost scales with the shortlist, not the universe: screening is free, so the debate runs on ten stocks however large the universe grows. Analysts run in parallel, each role can use its own Foundry deployment, calls are capped at four per deployment, and Redis caching means repeated inputs cost nothing. DuckDB scans 1.27 million bars in place, and the API is stateless behind PostgreSQL and Redis, so it can be scaled out without code changes.

12.5 REAL-WORLD APPLICABILITY

The platform runs in production on Azure and connects to Zerodha Kite through a self-hosted broker gateway, with a kill switch, repeat-safe orders and reconciliation in place. Paper mode is the default, live trading needs an explicit confirmation flag, and the white-box order logic fits SEBI's framework for retail algorithmic trading.

13. CONCLUSION AND FUTURE WORK

13.1 CONCLUSION

This project designed, built and deployed an evidence-first multi-agent AI system for NSE equity research. Seven agents read and argue over dated evidence; deterministic code recomputes every trade after charges and has the final say. Tested honestly, the best strategy

returned +134.22% on unseen data with an efficiency of 0.95, the fixed candidate finished 9.75 points ahead of NIFTY 50 on an untouched holdout, and Jev's calibrated questions lifted the hit rate from 22% to 31%. The result is a system whose numbers can be defended: the model advises, and the code decides.

13.2 FUTURE WORK

- **Paper-trading phase:** run the fixed candidate for three months on the live broker in paper mode and compare its Sharpe with the backtest.
- **Jev v4 questions:** sector-aware questions, judged on stocks never used in design.
- **Wider fundamentals:** extend dated fundamentals from 284 companies to the full screened universe.
- **Model diversity:** different model families for the bull and the bear, so they do not share the same mistakes.

REFERENCES

- [1] Y. Xiao, E. Sun, D. Luo and W. Wang, "TradingAgents: Multi-Agents LLM Financial Trading Framework," arXiv:2412.20138, 2024.
- [2] D. H. Bailey and M. López de Prado, "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality," *Journal of Portfolio Management*, vol. 40, no. 5, pp. 94–107, 2014.
- [3] D. H. Bailey, J. Borwein, M. López de Prado and Q. J. Zhu, "The Probability of Backtest Overfitting," *Journal of Computational Finance*, vol. 20, no. 4, pp. 39–69, 2017.
- [4] R. Pardo, *The Evaluation and Optimization of Trading Strategies*, 2nd ed. Hoboken, NJ: Wiley, 2008.
- [5] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems* 33, 2020.
- [6] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum and I. Mordatch, "Improving Factuality and Reasoning in Language Models through Multiagent Debate," arXiv:2305.14325, 2023.
- [7] Securities and Exchange Board of India, "Safer participation of retail investors in Algorithmic trading," SEBI circular, 4 February 2025.
- [8] National Stock Exchange of India, "Historical Market Data and Index Methodologies," <https://www.nseindia.com>, 2026.
- [9] Zerodha Broking Ltd., "Brokerage and statutory charges for Indian equities," <https://zerodha.com/charges>, 2026.
- [10] M. Harish and M. Afrah, "Trading Agent: Evidence-First Multi-Agent AI for NSE Equity Research," source code, <https://github.com/HarishM-AIML-A-58/Trading-Agent>, Oct. 2026.